

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет безопасности информационных технологий

Дисциплина:
«Информационная безопасность баз данных»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4
«Разграничение доступа»

Выполнил:

Пахомов Владимир Юрьевич, студент группы N3250



(подпись)

Проверил:

Ярцева Наталия Андреевна, преподаватель ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026г.

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ.....	2
ВВЕДЕНИЕ.....	3
ХОД РАБОТЫ.....	3
Задание 1.....	3
Задание 2.....	4
Задание 3.....	4
Задание 4.....	5
Задание 5.....	5
Задание 6.....	6
Задание 7.....	6
Задание 8.....	8
ЗАКЛЮЧЕНИЕ.....	9

ВВЕДЕНИЕ

Цель работы — освоить методы разграничения прав доступа и аудита действий пользователей в PostgreSQL.

ХОД РАБОТЫ

В данной лабораторной работе для взаимодействия с базой данных PostgreSQL будем использовать утилиту psql на ОС Linux. Для создания и подключения к локальной базе данных создадим файл lab4.sql и впишем в него все команды для создания базы данных и отправки команд из методического материала. Для подключения к базе данных используем команду:

```
psql -U postgres -f lab4.sql
```

Для начала создадим четыре пользователя:

```
CREATE USER user1 WITH PASSWORD 'passwd1';  
CREATE USER user2 WITH PASSWORD 'passwd2';  
CREATE USER user3 WITH PASSWORD 'passwd3';  
CREATE USER user4 WITH PASSWORD 'passwd4';
```

Задание 1

Создадим две простые таблицы для дальнейшей работы с правами доступа:

```
CREATE TABLE departments (  
    dept_id SERIAL PRIMARY KEY,  
    dept_name TEXT NOT NULL  
);  
  
CREATE TABLE employees (  
    emp_id SERIAL PRIMARY KEY,  
    name TEXT NOT NULL,  
    salary NUMERIC,  
    dept_id INTEGER REFERENCES departments(dept_id),  
    created_by TEXT DEFAULT current_user  
);
```

Задание 2

Выдаём нужные права созданным ранее пользователям:

```
GRANT ALL PRIVILEGES ON TABLE departments TO user1;
GRANT ALL PRIVILEGES ON TABLE employees TO user1;

GRANT INSERT, SELECT, UPDATE ON TABLE departments TO user2;
GRANT INSERT, SELECT, UPDATE ON TABLE employees TO user2;

GRANT DELETE ON TABLE departments TO user3 WITH GRANT OPTION;
GRANT DELETE ON TABLE employees TO user3 WITH GRANT OPTION;
```

Задание 3

С помощью пользователя user3 выдадим пользователю user4 право на удаление данных из таблиц:

```
SET ROLE user3;
GRANT DELETE ON TABLE departments TO user4;
GRANT DELETE ON TABLE employees TO user4;
RESET ROLE;
```

Проверим все выданные права:

Schema	Name	Type	Access privileges		Column privileges	Policies
			Access privileges			
public	employees	table	postgres=arwdDxtm/postgres+			
			user1=arwdDxtm/postgres			
			user2=arw/postgres			
			user3=d*/postgres			
			user4=d/user3			
(1 row)						

Schema	Name	Type	Access privileges		Column privileges	Policies
			Access privileges			
public	departments	table	postgres=arwdDxtm/postgres+			
			user1=arwdDxtm/postgres			
			user2=arw/postgres			
			user3=d*/postgres			
			user4=d/user3			

Задание 4

Отзовём все ранее выданные права у всех пользователей:

```
REVOKE ALL PRIVILEGES ON TABLE departments, employees FROM user1;
REVOKE INSERT, SELECT, UPDATE ON TABLE departments, employees FROM user2;
REVOKE DELETE ON TABLE employees, departments FROM user3 CASCADE;
```

Проверка:

Schema	Name	Type	Access privileges	Column privileges	Policies
public	employees	table	postgres=arwdDxtm/postgres		

(1 row)

Schema	Name	Type	Access privileges	Column privileges	Policies
public	departments	table	postgres=arwdDxtm/postgres		

(1 row)

Задание 5

Создадим две схемы для двух пользователей и от лица каждого пользователя создадим по одной таблице в схемах:

```
CREATE SCHEMA subschema_user1 AUTHORIZATION user1;
CREATE SCHEMA subschema_user2 AUTHORIZATION user2;
```

```
SET ROLE user1;
CREATE TABLE subschema_user1.work_tasks (
    task_id SERIAL PRIMARY KEY,
    description TEXT,
    deadline DATE
);
RESET ROLE;
```

```
SET ROLE user2;
CREATE TABLE subschema_user2.sales_reports (
    report_id SERIAL PRIMARY KEY,
    amount NUMERIC,
    region TEXT
);
RESET ROLE;
```

Теперь чтобы убедиться в выданных правах, попробуем будучи пользователем user1 получить доступ к своей таблице и к чужой:

```
SET ROLE user1;
SELECT * FROM subschema_user1.work_tasks;
RESET ROLE;
```

```

task_id | description | deadline
-----+-----+-----
(0 rows)

```

```

SET ROLE user1;
SELECT * FROM subschema_user2.sales_reports;
RESET ROLE;

```

```

psql:lab4.sql:84: ERROR: permission denied for schema subschema_user2
LINE 1: SELECT * FROM subschema_user2.sales_reports;

```

Как мы видим при попытке прочитать чужую таблицу мы получаем ошибку.

Задание 6

Создадим представление и предоставим доступ на чтение только пользователю user2:

```

CREATE VIEW employees_info AS
SELECT e.name, d.dept_name
FROM employees AS e
JOIN departments AS d ON e.dept_id = d.dept_id;

GRANT SELECT ON employees_info TO user2;

```

По итогу доступ к представлению выглядит следующим образом:

Schema	Name	Type	Access privileges	Column privileges	Policies
			Access privileges		
public	employees_info	view	postgres=arwdBxtm/postgres+ user2=r/postgres		

(1 row)

Задание 7

Перед тем как создавать политику безопасности на уровне строк добавим пару записей от пользователей в таблицу employees:

```

GRANT SELECT, INSERT ON employees TO user1, user2;
GRANT USAGE, SELECT ON SEQUENCE employees_emp_id_seq TO user1, user2;
SET ROLE user1;
INSERT INTO employees (name, salary, dept_id, created_by)
VALUES ('Pakhomov', 100000, 1, 'user1');
RESET ROLE;
SET ROLE user2;
INSERT INTO employees (name, salary, dept_id, created_by)
VALUES ('Brown', 90000, 2, 'user2');
RESET ROLE;

```

Далее создадим политику ROL:

```
ALTER TABLE employees ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY user_row_policy
ON employees
FOR SELECT
TO public
USING (created_by = current_user);
```

Теперь при попытке чтения строк из таблицы employees пользователи будут видеть только те строки, которые имеют поле created_by, совпадающее с именем пользователя:

emp_id	name	salary	dept_id	created_by
1	Ivanov	50000	1	postgres
2	Petrov	60000	1	postgres
3	Sidorov	45000	2	postgres
4	Kuznetsov	55000	3	postgres
5	Pakhomov	100000	1	user1
6	Brown	90000	2	user2

(6 rows)

```
SET user1
```

emp_id	name	salary	dept_id	created_by
5	Pakhomov	100000	1	user1

(1 row)

```
RESET
```

```
SET user2
```

emp_id	name	salary	dept_id	created_by
6	Brown	90000	2	user2

(1 row)

```
RESET
```

				Access privileges		Policies	
Schema	Name	Type		Access privileges	Column privileges		
public	employees	table		postgres=arwdDxtm/postgres+		user_row_policy (r):	
				user1=ar/postgres	+	(u): (created_by =	
				user2=ar/postgres			

(1 row)

Задание 8

Для начала создадим таблицу с логами и выдадим права пользователям на взаимодействие (SELECT, INSERT, UPDATE, DELETE) с таблицей employees:

```
CREATE TABLE logs (  
    log_id SERIAL PRIMARY KEY,  
    table_name TEXT,  
    operation TEXT,  
    user_name TEXT,  
    old_data JSONB,  
    new_data JSONB,  
    changed_at TIMESTAMP DEFAULT now()  
);  
  
GRANT SELECT, INSERT, UPDATE, DELETE ON employees TO user1, user2;  
GRANT USAGE, SELECT ON SEQUENCE employees_emp_id_seq TO user1, user2;
```

Далее создадим триггер и функцию к нему. Триггер будет реагировать на изменения в таблице employees и записывать лог в таблицу logs. При этом пользователи не будут иметь доступ к таблице логов, запись в неё будет происходить от лица админа postgres:

```
CREATE OR REPLACE FUNCTION logs_trigger() RETURNS trigger AS $$  
BEGIN  
    IF (TG_OP = 'INSERT') THEN  
        INSERT INTO logs (table_name, operation, user_name, new_data)  
        VALUES (TG_TABLE_NAME, TG_OP, current_user, row_to_json(NEW)::jsonb);  
    ELSIF (TG_OP = 'UPDATE') THEN  
        INSERT INTO logs (table_name, operation, user_name, old_data, new_data)  
        VALUES (TG_TABLE_NAME, TG_OP, current_user, row_to_json(OLD)::jsonb,  
row_to_json(NEW)::jsonb);  
    ELSIF (TG_OP = 'DELETE') THEN  
        INSERT INTO logs (table_name, operation, user_name, old_data)  
        VALUES (TG_TABLE_NAME, TG_OP, current_user, row_to_json(OLD)::jsonb);  
    END IF;  
    RETURN NULL;  
END;  
$$ LANGUAGE plpgsql  
SECURITY DEFINER;  
  
CREATE TRIGGER trigger_employees_logs  
AFTER INSERT OR UPDATE OR DELETE ON employees  
FOR EACH ROW EXECUTE FUNCTION logs_trigger();
```

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены и практически реализованы основные механизмы обеспечения безопасности и контроля доступа в СУБД PostgreSQL.

В процессе работы были освоены следующие ключевые технологии:

- Использование команд GRANT и REVOKE для выдачи прав, а также реализация цепочки делегирования полномочий;
- Создание персональных подсхем с авторизацией владельца, что позволяет разграничить рабочие пространства пользователей внутри одной базы данных;
- На уровне столбцов был реализован доступ через представление, на уровне строк - при помощи политики RLS;
- Разработан автоматизированный механизм логирования изменений на основе триггеров и триггерных функций. Это позволило обеспечить фиксацию всех операций с сохранением истории изменений.

Изученные инструменты позволяют создать гибкую и надёжную систему защиты данных, обеспечивающую не только разграничение прав, но и полную прозрачность действий пользователей, что критически важно для защиты коммерческой и конфиденциальной информации в современных СУБД.